EXP NO.8

Enable real-time communication via WebSockets

(To implement real-time communication between clients and a server using WebSockets in a full-stack web application.)

The real-time chat application was successfully implemented using React, Node.js, Express, and Socket.IO. Messages were exchanged instantly between connected clients without refreshing the browser.

(Important: This is a basic demonstration. Messages are not stored permanently, and the username is not authenticated. For a production application, add proper authentication, input validation, rate limiting, and HTTPS.)

Software requirements:

- Windows operating system
- Visual Studio Code
- Node.js and npm
- Web browser
- Internet connection for installing packages

We will create a simple real-time chat application using React, Node.js, Express, and Socket.IO. When one student sends a message, it appears immediately on the other connected browser without refreshing the page.

Step 1: Create the project

Open VS Code and select **Terminal → New Terminal.**

Execute:

```bash
mkdir websocket-chat
cd websocket-chat
mkdir backend
```

Create this folder structure:

```
websocket-chat
    backend/
        server.js
    frontend/ (created in Step 4)
```

Step 2: Set up the backend

In the terminal, execute:

```bash
cd backend
npm init -y
npm install express socket.io cors
```

Expected output: npm installs the Express, Socket.IO, and CORS packages and creates `package.json`.

Create a file named `server.js` inside the `backend` folder.

Paste the following code:

JavaScript

```
const express = require("express");
const http = require("http");
const cors = require("cors");
const { Server } = require("socket.io");

const app = express();
app.use(cors());

const server = http.createServer(app);

const io = new Server(server, {
  cors: {
    origin: "http://localhost:5173"
  }
});
```

```
  console.log("A user connected:", socket.id);

  socket.on("chatMessage", (message) => {
    io.emit("chatMessage", message);
  });

  socket.on("disconnect", () => {
    console.log("A user disconnected");
  });
});

server.listen(5000, () => {
  console.log("WebSocket server running on port 5000");
});
```

Explanation:

- `connection` : detects when a client connects.
- `chatMessage` : receives a message from a client.
- `io.emit()` : broadcasts the message to all connected clients.
- `disconnect` : detects when a client leaves.

Step 3: Understand the backend output

Open a new terminal in VS Code and run the backend:

```bash
cd websocket-chat/backend
node server.js
```

Expected terminal output:

```
WebSocket server running on port 5000
```

Keep this terminal running. The server is now ready to accept connections.

Step 4: Create the React frontend

Open a **second terminal** in VS Code. Return to the main project folder and create the React application:

```bash
cd websocket-chat
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm install socket.io-client
```

Expected output: The React project is created and the Socket.IO client library is installed.

Step 5: Write the frontend code

Open `frontend/src/App.jsx`, remove the existing code, and paste the following:

```
import { useEffect, useState } from "react";
import { io } from "socket.io-client";
import "./App.css";
```

```
function App() {
  const [username, setUsername] = useState("");
  const [message, setMessage] = useState("");
  const [messages, setMessages] = useState([]);
  const [connected, setConnected] = useState(false);

  useEffect(() => {
    const socket = io("http://localhost:5000");

    socket.on("connect", () => setConnected(true));
    socket.on("disconnect", () => setConnected(false));

    socket.on("chatMessage", (data) => {
      setMessages((previous) => [...previous, data]);
    });

    return () => socket.disconnect();
  }, []);

  const sendMessage = (event) => {
    event.preventDefault();

    if (!username.trim() || !message.trim()) {
      return;
    }

    // Send the message through Socket.IO
    // The server broadcasts it to every connected client.
    const socketMessage = {
      username: username.trim(),
      message: message.trim()
    };
```

```
  // Use the shared socket connection in the next step.
};

return (
  <div className="chat-container">
    <h1>Real-Time Chat Application</h1>

    <p className={connected ? "online" : "offline"}>
      {connected ? "Connected" : "Connecting..."}
    </p>

    <input
      placeholder="Enter your name"
      value={username}
      onChange={(e) => setUsername(e.target.value)}
    />

    <div className="messages">
      {messages.map((item, index) => (
        <p key={index}>
          <strong>{item.username}:</strong> {item.message}
        </p>
      ))}
    </div>

    <form onSubmit={sendMessage}>
      <input
        placeholder="Type a message"
        value={message}
        onChange={(e) => setMessage(e.target.value)}
      />
```

```
        <button type="submit">Send</button>
      </form>
    </div>
  );
}

export default App;
```

Step 6: Complete the message-sending functionality

To share the same Socket.IO connection between the connection listener and the send function, make these two changes in `App.jsx`.

Change 1: Update the import statement:

JavaScript

```
import { useEffect, useState, useRef } from "react";
```

Change 2: Inside `App()`, below the state declarations, add:

JavaScript

```
const socketRef = useRef(null);
```

Inside `useEffect`, replace:

JavaScript

```
const socket = io("http://localhost:5000");
```

with:

JavaScript

```
const socket = io("http://localhost:5000");
socketRef.current = socket;
```

Then replace the `// Use the shared socket connection in the next step.` comment in `sendMessage` with:

JavaScript

```
socketRef.current.emit("chatMessage", socketMessage);
setMessage("");
```

Step 7: Design the chat interface

Open `frontend/src/App.css`, remove the existing content, and paste:

CSS

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #eef2f7;
}

.chat-container {
  max-width: 600px;
  margin: 40px auto;
  padding: 25px;
  background: white;
  border-radius: 12px;
  box-shadow: 0 4px 12px #00000020;
}
```

```css
h1 {
  text-align: center;
  color: #1d4ed8;
}

.online {
  color: green;
  text-align: center;
}

.offline {
  color: red;
  text-align: center;
}

input {
  box-sizing: border-box;
  width: 100%;
  padding: 12px;
  margin: 8px 0;
  border: 1px solid #ccc;
  border-radius: 6px;
}
```

```css
.messages {
  min-height: 180px;
  max-height: 300px;
  overflow-y: auto;
  margin: 15px 0;
  padding: 10px;
  background: #f8fafc;
  border: 1px solid #ddd;
  border-radius: 6px;
}

button {
  padding: 12px 20px;
  background: #2563eb;
  color: white;
  border: none;
  border-radius: 6px;
  cursor: pointer;
}

button:hover {
  background: #1d4ed8;
}
```

Step 8: Run the frontend

In the second terminal, ensure that you are inside the `frontend` folder and execute:

```bash
npm run dev
```

Expected terminal output:

```
VITE ready
Local: http://localhost:5173/ ↗
```

Open the application:

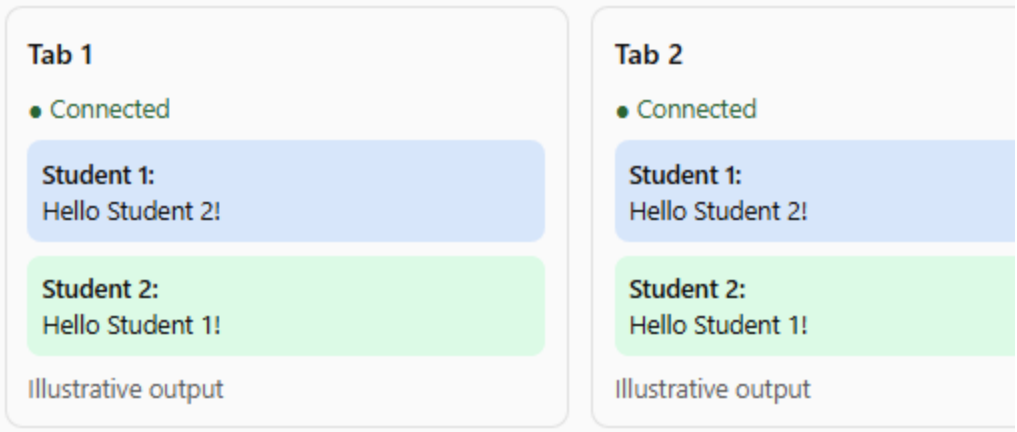
Open the Real-Time Chat Application ↗

The status should display **Connected**.

Step 9: Test real-time communication

1. Open the application in two separate browser tabs.
2. In Tab 1, enter the username `Student 1`.
3. In Tab 2, enter the username `Student 2`.
4. In Tab 1, type `Hello Student 2!` and click **Send**.
5. Observe Tab 2. The message should appear immediately.
6. Reply from Tab 2 and verify that the reply appears in Tab 1.

Expected browser output



Both tabs display the same messages because the server broadcasts each message to all connected clients.

Step 10: Verify the server output

Check the backend terminal. It should display messages similar to:

```
WebSocket server running on port 5000
A user connected: <socket-id>
A user connected: <socket-id>
```

The actual socket IDs will be different.